

```
%%writefile utils.py

def plot_data(data, title='Data Plot', xlabel='X-axis', ylabel='Y-axis'):
    import matplotlib.pyplot as plt
    plt.figure(figsize=(10, 6))
    plt.plot(data)
    plt.title(title)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.grid(True)
    plt.show()
```

Writing utils.py

After running the above cell, a file named `utils.py` will be created in your Colab environment. You can verify its presence using the file browser on the left panel or by running `!ls` in a new code cell. Once the file is present, your original import statement should work.

```
import numpy as np
import matplotlib.pyplot as plt
from utils import plot_data
```

```
input_array = np.array([1,2,3])
exp_array = np.exp(input_array)

print("Input to exp:", input_array)
print("Output of exp:", exp_array)
```

```
input_val = 1
exp_val = np.exp(input_val)

print("Input to exp:", input_val)
print("Output of exp:", exp_val)
```

```
Input to exp: [1 2 3]
Output of exp: [ 2.71828183  7.3890561  20.08553692]
Input to exp: 1
Output of exp: 2.718281828459045
```

```
def sigmoid(z):
    """
    Compute the sigmoid of z

    Args:
        z (ndarray): A scalar, numpy array of any size.
```

```

Returns:
    g (ndarray): sigmoid(z), with the same shape as z

"""

g = 1/(1+np.exp(-z))

return g

```

```

z_tmp = np.arange(-10,11)
y = sigmoid(z_tmp)
np.set_printoptions(precision=3)
print("Input (z), Output (sigmoid(z))")
print(np.c_[z_tmp, y])

```

```

Input (z), Output (sigmoid(z))
[[-1.000e+01  4.540e-05]
 [-9.000e+00  1.234e-04]
 [-8.000e+00  3.354e-04]
 [-7.000e+00  9.111e-04]
 [-6.000e+00  2.473e-03]
 [-5.000e+00  6.693e-03]
 [-4.000e+00  1.799e-02]
 [-3.000e+00  4.743e-02]
 [-2.000e+00  1.192e-01]
 [-1.000e+00  2.689e-01]
 [ 0.000e+00  5.000e-01]
 [ 1.000e+00  7.311e-01]
 [ 2.000e+00  8.808e-01]
 [ 3.000e+00  9.526e-01]
 [ 4.000e+00  9.820e-01]
 [ 5.000e+00  9.933e-01]
 [ 6.000e+00  9.975e-01]
 [ 7.000e+00  9.991e-01]
 [ 8.000e+00  9.997e-01]
 [ 9.000e+00  9.999e-01]
 [ 1.000e+01  1.000e+00]]

```

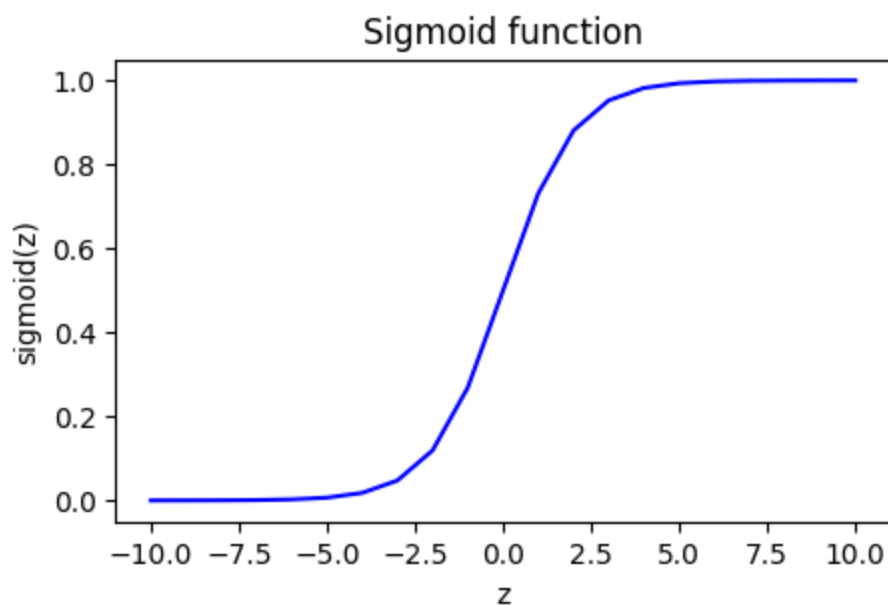
```

fig,ax = plt.subplots(1,1,figsize=(5,3))
ax.plot(z_tmp, y, c="b")

ax.set_title("Sigmoid function")
ax.set_ylabel('sigmoid(z)')
ax.set_xlabel('z')

```

Text(0.5, 0, 'z')



```
X = np.array([[0.5, 1.5], [1,1], [1.5, 0.5], [3, 0.5], [2, 2], [1, 2.5]])
y = np.array([0, 0, 0, 1, 1, 1]).reshape(-1,1)
```

```
x0 = np.arange(0,6)
x1 = 3 - x0
fig,ax = plt.subplots(1,1,figsize=(5,4))
ax.plot(x0,x1, c="b")
ax.axis([0, 4, 0, 3.5])
ax.fill_between(x0,x1, alpha=0.2)
plot_data(X,y,ax)
ax.set_ylabel(r'$x_1$')
ax.set_xlabel(r'$x_0$')
plt.show()
```



```
from sklearn.linear_model import LogisticRegression
lr_model = LogisticRegression()
lr_model.fit(X, y)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:1408: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples, 1), for example using y = column_or_1d(y, warn=True)
```

▼ LogisticRegression ⓘ ?
LogisticRegression()

```
1.5  
y_pred = lr_model.predict(X)  
print("Prediction on training set:", y_pred)
```

```
Prediction on training set: [0 0 0 1 1 1]
```

```
0.0  
print("Accuracy on training set:", lr_model.score(X, y))
```

```
Accuracy on training set: 1.0
```

```
[[0]  
[0]  
[1]]
```

```
print(f"Theta/Parameters of the model: w0 = {lr_model.coef_[0][0]}, w1 = {lr_model.coef_[0][1]}")
```

```
Theta/Parameters of the model: w0 = 0.9041134943596743, w1 = 0.7358754294504456,
```

Start coding or [generate](#) with AI.